

Documento de Arquitetura de Software

GRUPO 06 – AGENDAWEB

Gestor do Projeto	Gerente de Projeto
Luiz Eduardo dos Anjos de Jesus	Luis Gustavo Lima Fernandes
luiz.anjos@a.ucb.br	luis.gfernandes@a.ucb.br
(61) 99111-2992	

Objetivo deste Documento

Este documento tem como objetivo descrever as principais decisões de projeto tomadas pela equipe de desenvolvimento e os critérios considerados durante a tomada destas decisões. Suas informações incluem a parte de *hardware* e *software* do sistema.

Histórico de Revisão

Data	Demanda	Autor	Descrição	Versão
02/05/2026	BETA-0001	Luiz Eduardo	Implementação inicial na web, com funções e layouts básicos para o início do projeto.	0.1.0
10/05/2026	BETA-0002	Luiz Eduardo	Melhoria na interface web, aplicação da arquitetura nas tecnologias escolhidas para o desenvolvimento e testes das api's restfull.	0.1.1
22/05/2026	BETA-0003	Luiz Eduardo	Aplicação e revisão dos conceitos criados através dos diagramas criados para o funcionamento do projeto.	0.2.0
25/05/2026	BETA-0004	Luís Gustavo	Configuração do banco de dados relacional e criação dos primeiros Models com o Sequelize (Usuários, Clientes).	0.2.1
27/05/2026	BETA-0005	Luís Gustavo	Implementação do sistema de login, controle de sessões de usuário (express-session) e middlewares de segurança.	0.2.2
31/05/2026	BETA-0006	Luís Gustavo e Luiz Eduardo	Desenvolvimento do painel administrativo (Dashboard) e conclusão do fluxo de CRUD (Criar, Ler, Atualizar, Deletar) para Serviços e Agendamentos.	0.3.0
08/06/2026 e 09/06/2026	BETA-0007	Luís Gustavo	Realização de testes de integração, ajustes finos na responsividade das telas (views em Mustache) e correções de bugs relatados no fluxo de agendamento.	0.3.1
17/06/2026	FINAL-0001	Luiz Eduardo	Preparação para a versão final da entrega. Revisão geral de código e documentação da arquitetura.	1.0.0

Sumário

1. INTRODUÇÃO	3
1.1 Finalidade.....	3
1.2 Escopo	3
1.3 Definições, Acrônimos e Abreviações	3
1.4 Referências	4
2. REPRESENTAÇÃO ARQUITETURAL.....	4
3. REQUISITOS E RESTRIÇÕES ARQUITETURAIS	5
4. VISÃO DE CASOS DE USO.....	6
4.1 Casos de Uso significantes para a arquitetura.....	10
5. VISÃO LÓGICA.....	10
5.1 Visão Geral – pacotes e camadas.....	10
6. VISÃO DE IMPLEMENTAÇÃO.....	12
6.1 Caso de Uso [00X]	12
6.1.1 Diagrama de Classes	12
6.1.2 Diagrama de Sequência	13
7. VISÃO DE IMPLANTAÇÃO.....	14
8. DIMENSIONAMENTO E PERFORMANCE.....	14
8.1 Volume.....	14
8.2 Performance.....	14
9. QUALIDADE	14

1. INTRODUÇÃO

1.1 Finalidade

Este documento fornece uma visão arquitetural abrangente do sistema AgendaWeb, usando diversas visões de arquitetura para **representar** diferentes aspectos do sistema. O objetivo deste documento é capturar e comunicar as decisões arquiteturais significativas que foram tomadas em relação ao sistema.

O documento irá adotar uma estrutura baseada na visão “4+1” de modelo de arquitetura [KRU41].



Figura 1 – Arquitetura 4+1

1.2 Escopo

Este Documento de Arquitetura de Software se aplica ao AgendaWeb, que será desenvolvido pelo GRUPO 06.

1.3 Definições, Acrônimos e Abreviações

QoS – Quality of Service, ou qualidade de serviço. Termo utilizado para descrever um conjunto de qualidades que descrevem os requisitos não-funcionais de um sistema, como performance, disponibilidade e escalabilidade[QOS].

1.4 Referências

[KRU41]: The “4+1” view model of software architecture, Philippe Kruchten, November 1995, <http://www3.software.ibm.com/ibmdl/pub/software/rational/web/whitepapers/2003/Pbk4p1.pdf>

[QOS] <https://docs.oracle.com/cd/E19636-01/819-2326/6n4kfe7dj/index.html>

MARTIN, Robert C. Clean Code: a handbook of agile software craftsmanship. Upper Saddle River: Prentice Hall, 2008.

UNIVERSIDADE CATÓLICA DE BRASÍLIA. Template da disciplina Arquitetura de Software. Professor Oscar. Material disponibilizado no Ambiente Virtual de Aprendizagem (AVA). Brasília: Universidade Católica de Brasília, 2026. Disponível em: <https://ava.catolica.edu.br/>. Acesso em: 22 mai. 2026.

Os diagramas e artefatos arquiteturais apresentados neste documento foram elaborados com base nos conteúdos ministrados em sala de aula e nos materiais disponibilizados no Ambiente Virtual de Aprendizagem (AVA) da disciplina de Arquitetura de Software, sob orientação do Professor Oscar.

GOOGLE. Gemini. Plataforma de inteligência artificial generativa para apoio à produção e refinamento de conteúdo. Mountain View: Google, 2026. Disponível em: <https://gemini.google.com/>. Acesso em: 10 jun. 2026.

OPENAI. ChatGPT. Modelo de inteligência artificial para geração e refinamento de conteúdo textual. San Francisco: OpenAI, 2026. Disponível em: <https://chatgpt.com/>. Acesso em: 10 jun. 2026.

YOUTUBE. Curso de Draw.io (diagrams.net) – playlist de apoio para modelagem e criação de diagramas. YouTube, [s.d.]. Disponível em: <https://www.youtube.com/playlist?list=PLx6gdu4s3nkf34QO3SKMmhN5f6naRucu8>. Acesso em: 01 jun. 2026.

DRAW.IO. Diagrams.net: ferramenta para criação de diagramas técnicos e UML. [S.l.]: JGraph Ltd., 2026. Disponível em: <https://www.drawio.com/>. Acesso em: 01 jun. 2026.

EDUARDO, LUIZ. Diagrama de Arquitetura do Sistema AgendaWeb. Diagrams.net (Draw.io), 2026. Disponível em: https://viewer.diagrams.net/?tags=%7B%7D&lightbox=1&highlight=0000ff&edit=_blank&layers=1&nav=1&title=DesignDrawIO.drawio&dark=auto#Uhttps%3A%2F%2Fdrive.google.com%2Fuc%3Fid%3D1pkb0OW0Zw1pghbFzGg02T6hU_TmtSBZT%26export%3Ddownload. Acesso em: 17 jun. 2026.

2. REPRESENTAÇÃO ARQUITETURAL

Este documento irá detalhar as visões baseado no modelo “4+1” [KRU41], utilizando como referência os modelos definidos na MDS. As visões utilizadas no documento serão:

Visão	Público	Área	Modelo da MDS
Lógica	Analistas	Realização dos Casos de Uso	
Processo	Integradores	Performance, Escalabilidade, Concorrência	

Implementação	Programadores	Componentes de Software	
Implantação	Gerência de Configuração	Nodos físicos	
Caso de Uso	Todos	Requisitos funcionais	
Dados	Especialistas em dados Administradores de dados	Persistência de dados	

Tabela 1 – Visões, Público, Área e Artefatos da MDS

3. REQUISITOS E RESTRIÇÕES ARQUITETURAIS

Esta seção descrever os requisitos de software e restrições que tem um impacto significativo na arquitetura.

Requisito	Solução
Linguagem	O projeto utiliza JavaScript como linguagem principal de programação no back-end (Node.js). A construção da interface (front-end) é renderizada no servidor através da engine de templates Mustache (mustache-express), combinada com páginas em HTML, CSS e scripts no lado do cliente (client-side) armazenados na pasta pública.
Plataforma	A aplicação é baseada no ambiente de execução Node.js, utilizando o framework Express.js para a criação do servidor web e gerenciamento de rotas. Trata-se de uma aplicação monolítica de servidor, podendo ser hospedada em provedores VPS (como AWS EC2, DigitalOcean) ou plataformas PaaS (como Render, Heroku) que suportem Node.js.
Segurança	A segurança e a autenticação são gerenciadas internamente. O controle de acesso e de usuários logados é feito utilizando cookies de sessão via a biblioteca express-session no servidor. A proteção de rotas restritas (como as de administração ou exclusão de dados) ocorre por meio de Middlewares customizados no Express (ex: autenticacaoAdmin), que interceptam a requisição e validam os privilégios do usuário antes de devolver a página.
Persistência	O mecanismo de persistência de dados adotado é o banco de dados relacional SQLite (um banco local armazenado no arquivo database.sqlite). O acesso, manipulação e criação das tabelas no banco de dados não são feitos com SQL puro, mas através do ORM (Object-Relational Mapper) Sequelize, utilizando "Models" definidos no

	código JavaScript.
Internacionalização (i18n)	Não há implementação de internacionalização. O projeto — tanto na interface visual (templates HTML) quanto nas lógicas de respostas do servidor — foca estritamente em um único idioma: o Português do Brasil (PT-BR).

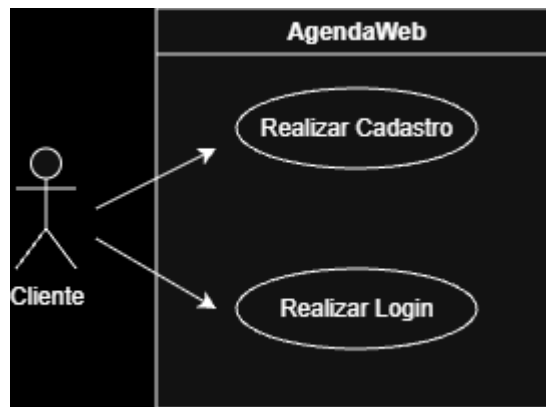
Tabela 2 – Exemplo de requisitos e restrições

4. VISÃO DE CASOS DE USO

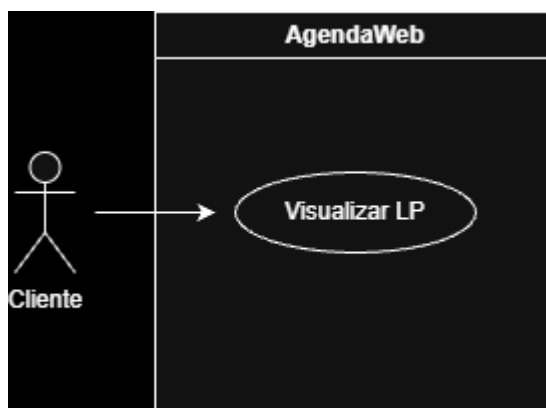
Esta seção lista as especificações centrais e significantes para a arquitetura do sistema.

Lista de casos de uso do sistema:

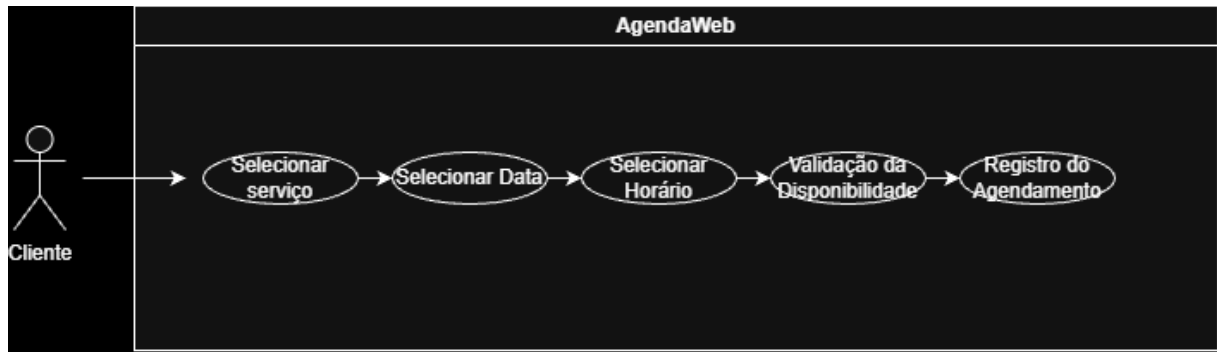
- **Caso de Uso 001 - Cadastro e Autenticação de Usuário**



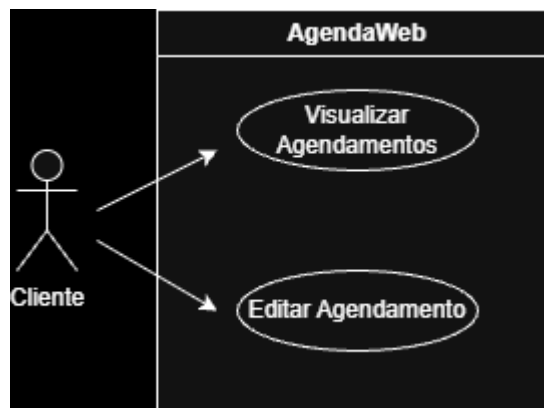
- **Caso de Uso 002 – Visualização de Landing Page**



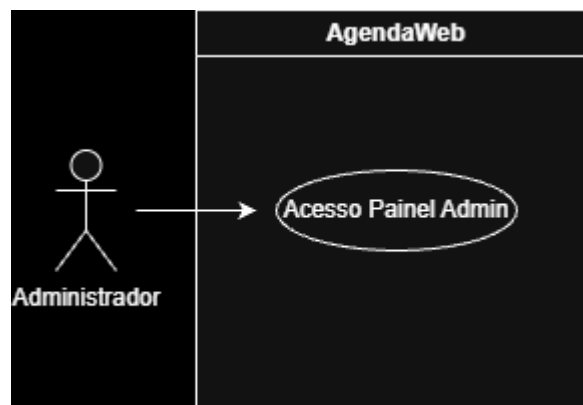
- **Caso de Uso 003 – Solicitação do Agendamento de Horário**



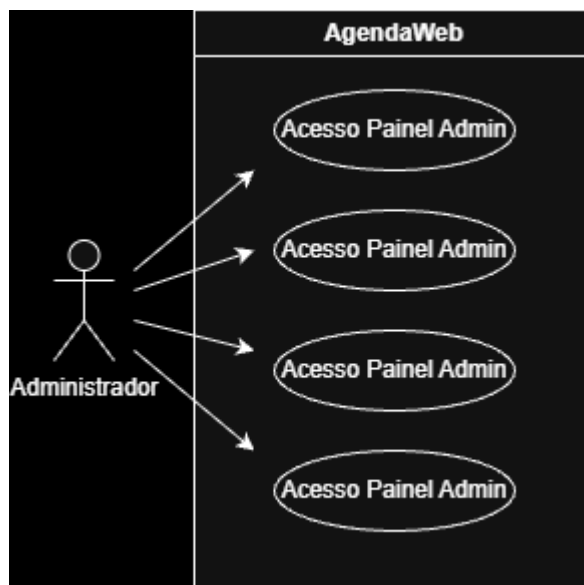
- **Caso de Uso 004 – Gerenciamento dos Meus Agendamentos**



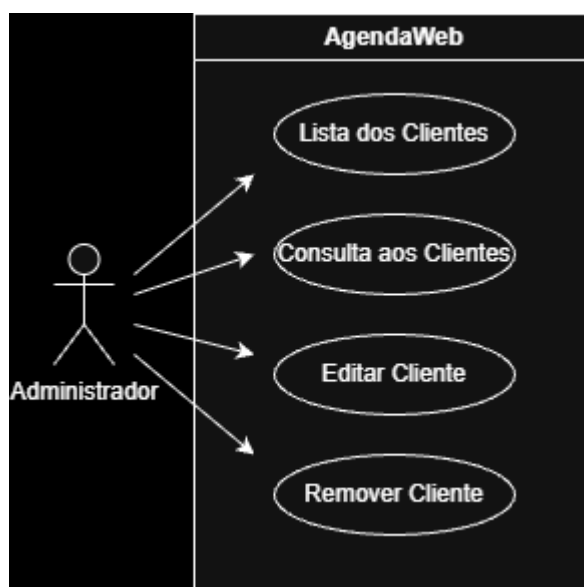
- **Caso de Uso 005 – Acesso ao Painel de Métricas**



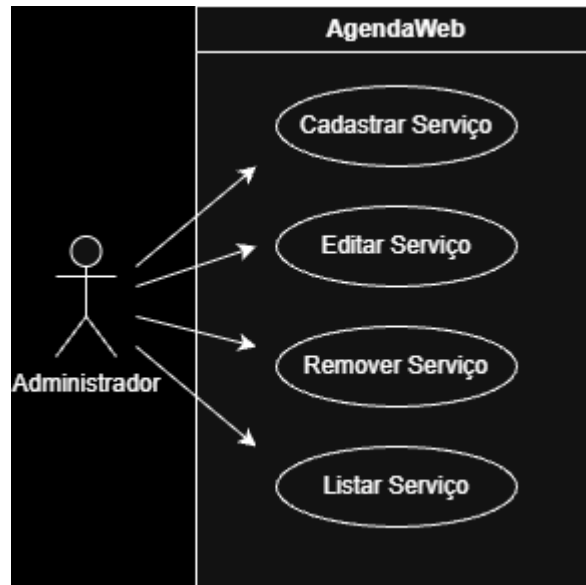
- **Caso de Uso 006 – Gerenciar Agendamentos Gerais**



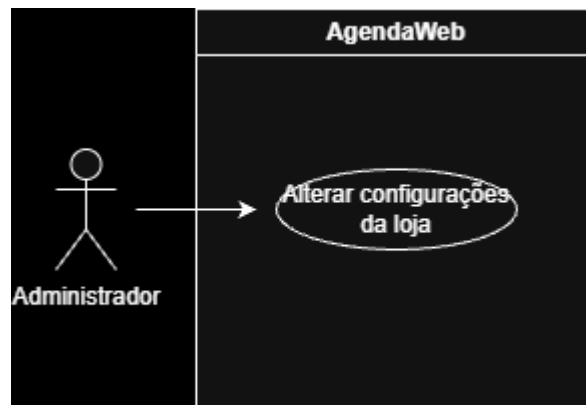
- Caso de Uso 007 – Gerenciamento de Clientes



- Caso de Uso 008 – Gerenciamento de Serviços



- Caso de Uso 009 – Gerenciamento da Loja



4.1 Casos de Uso significantes para a arquitetura

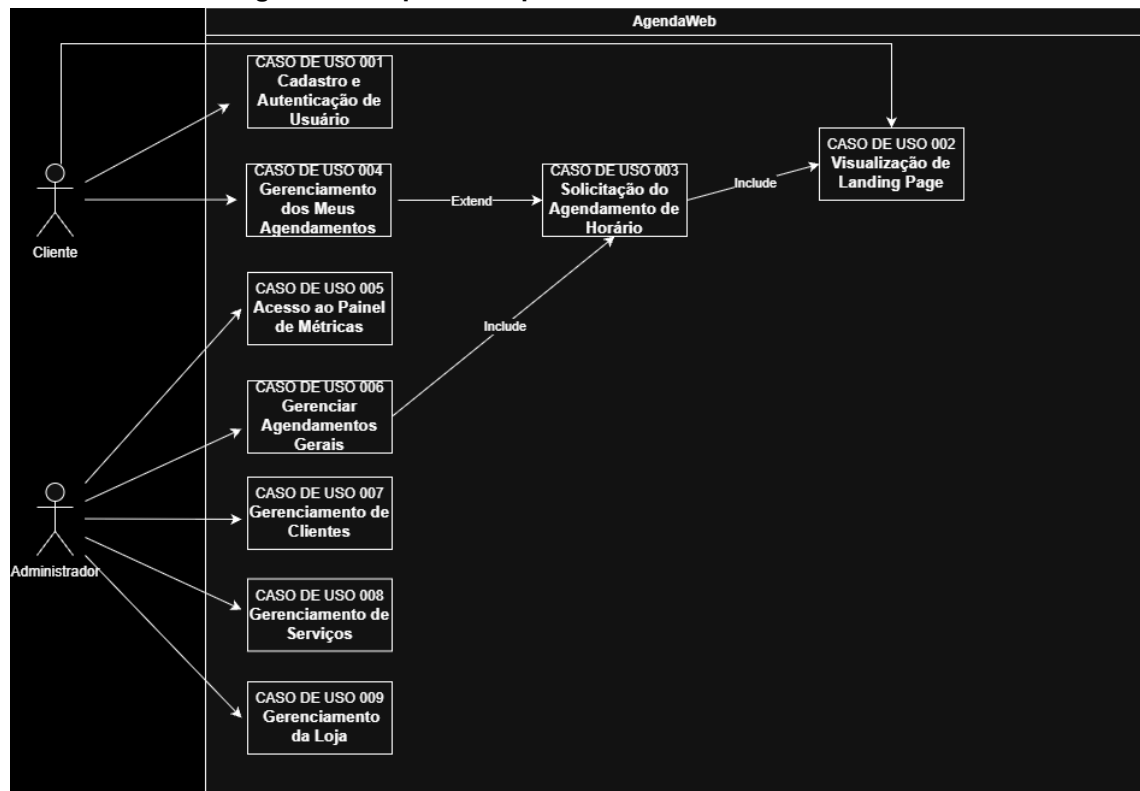


Figura 2 – Exemplo de Diagrama com os casos de uso significantes e atores

5. VISÃO LÓGICA

Descrever uma visão lógica da arquitetura. Descrever as classes mais importantes, sua organização em pacotes de serviços e subsistemas, e a organização desses subsistemas em camadas. Também descreve as realizações dos casos de uso mais importantes, por exemplo, aspectos dinâmicos da arquitetura. Diagramas de classes e sequência devem ser incluídos para ilustrar os relacionamentos entre as classes significativas na arquitetura, subsistemas, pacotes e camadas.

5.1 Visão Geral – pacotes e camadas

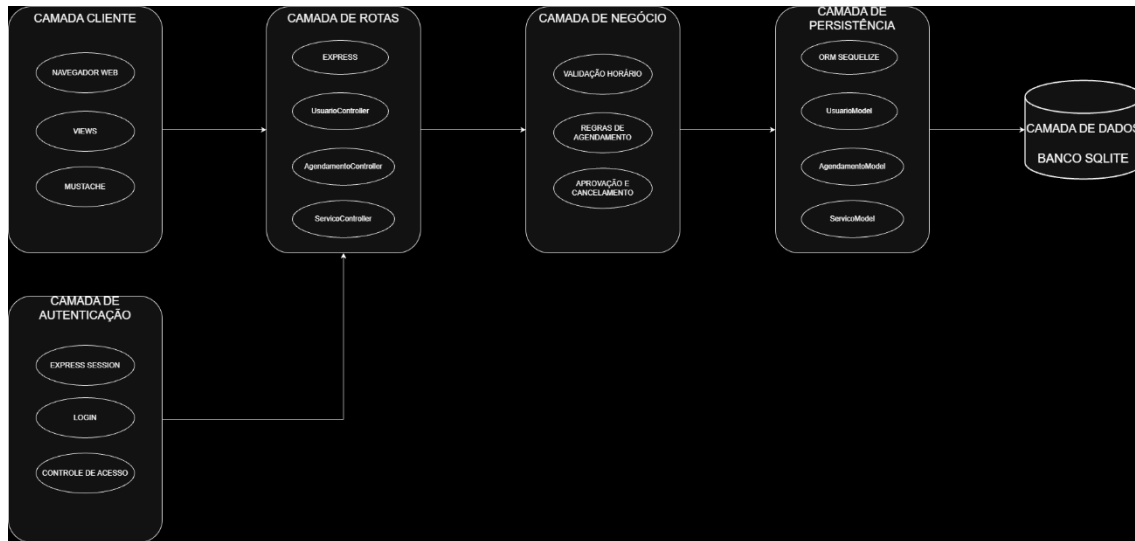


Figura 2 – Exemplo de Diagrama de Camadas da Aplicação

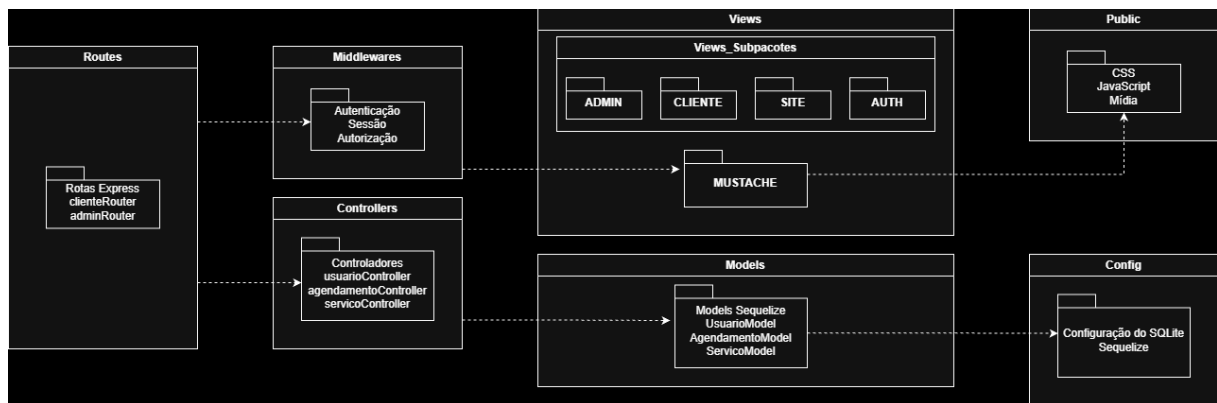


Figura 3 – Exemplo de Diagrama de Pacotes da Aplicação

6. VISÃO DE IMPLEMENTAÇÃO

6.1 Caso de Uso

6.1.1 Diagrama de Classes

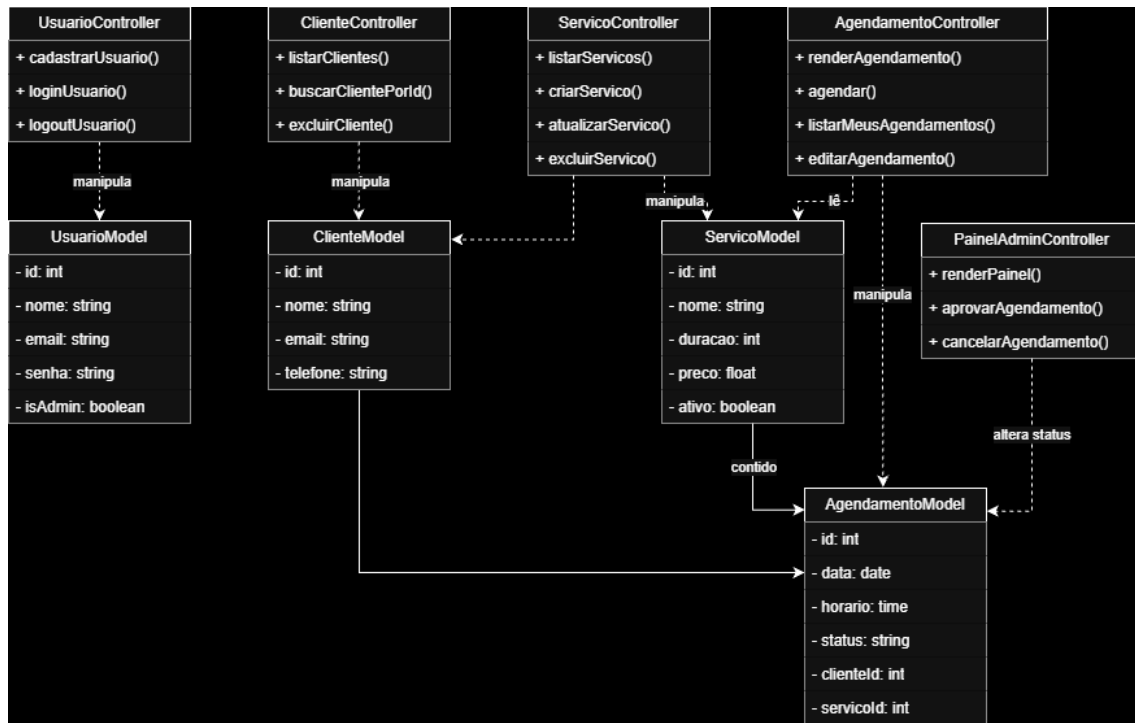


Figura 20 – Exemplo de Diagrama de Classes

6.1.2 Diagrama de Sequência

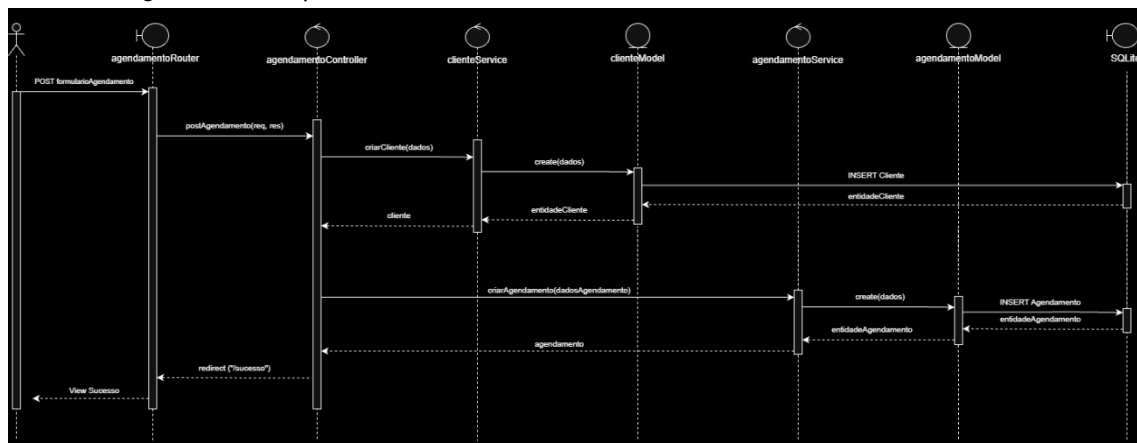


Figura 20 – Exemplo de Diagrama de Sequência

7. VISÃO DE IMPLANTAÇÃO

Descrever os nodos físicos, as configurações e os artefatos que serão implantados.

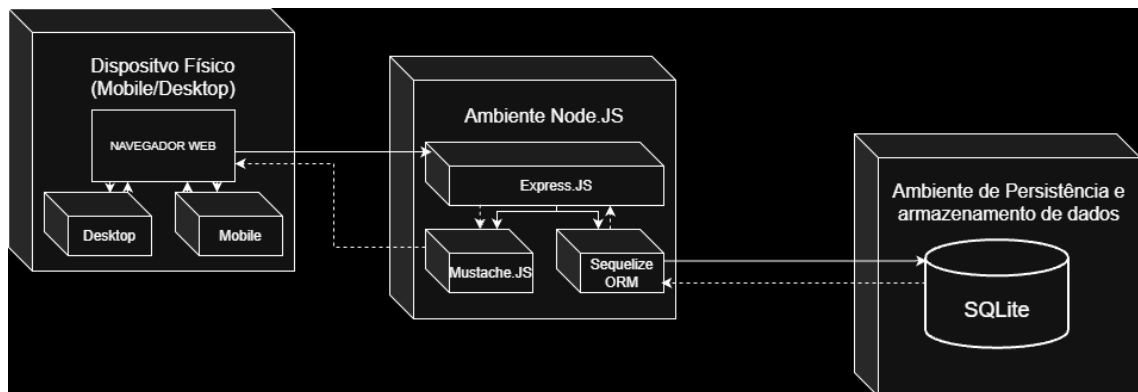


Figura 20 – Exemplo de Diagrama de Implantação

8. DIMENSIONAMENTO E PERFORMANCE

8.1 Volume

Enumerar os itens relativos ao volume de acesso aos recursos da aplicação:

- Número de estimado usuários: 500
- Número estimado de acessos diários: 173
- Número estimado de acessos por período: 4780/Mês
- Tempo de sessão de um usuário: Média de 15 minutos

8.2 Performance

Enumerar os itens referentes à resposta esperada do sistema:

- Tempo máximo para transações simples como login, navegação no calendário e consultas simples: 2 segundos.
- Tempo máximo para transações médias como efetuar agendamento e salvar dados de um cadastro: 3 a 5 segundos.
- Tempo máximo para transações complexas como gerar um relatório mensal, exportar dados e busca através de filtros(utilizando muita filtragem): 10 segundos.
- Tempo máximo geral: 10 segundos

9. QUALIDADE

Enumerar os itens de qualidade de software [QOS] significativos para a aplicação:

Item	Descrição	Solução
Escalabilidade	A capacidade de o sistema se adaptar e	O projeto adota uma arquitetura monolítica leve baseada em Node.js e Express. A

	suportar um aumento repentino no número de usuários ou transações sem perder desempenho.	escalabilidade do servidor pode ser feita com balanceadores de carga e gerenciadores (como PM2). O banco de dados atual (SQLite) atende bem a volumes moderados, mas graças ao uso do ORM (Sequelize), o sistema tem flexibilidade para escalar migrando facilmente o banco para PostgreSQL ou MySQL caso o volume de dados cresça.
Confiabilidade, Disponibilidade	A garantia de que a aplicação estará sempre online e acessível, além de resiliente contra falhas de infraestrutura (alto uptime).	O sistema possui poucos pontos de falha externos por ser independente de APIs de terceiros para o seu funcionamento principal. Além disso, por utilizar o banco de dados baseado em arquivo (SQLite), a rotina de backup e restauração é extremamente simples (basta copiar o arquivo database.sqlite), garantindo rápida recuperação do sistema em caso de falhas no servidor.
Portabilidade	A facilidade e capacidade de o código e os dados do sistema serem movidos de um provedor de nuvem, servidor ou infraestrutura para outro.	O sistema é altamente portátil. Sendo desenvolvido inteiramente em Node.js e utilizando um banco de dados embutido (SQLite), a aplicação não sofre de "vendor lock-in". Ela pode rodar em praticamente qualquer sistema operacional ou provedor de nuvem (Windows, Linux, VPS, contêineres Docker, Render, etc.) sem a necessidade de instalar um servidor de banco de dados separado.
Segurança	A proteção do sistema e das informações armazenadas contra acessos indevidos, interceptação de dados ou ataques.	O sistema realiza controle de acesso de forma nativa utilizando sessões no servidor (express-session). A segurança das rotas e das ações sensíveis (como o painel de controle) é garantida por Middlewares (como o autenticacaoAdmin), que bloqueiam o acesso de usuários não logados ou sem privilégios de administrador antes que eles atinjam as rotas protegidas.